HW8
my uni:sn3136

$$IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB$$

1.

(a) `addi $s1, $s2, 100`

- The value in `$s2` is read in **ID**
- The immediate value `100` comes from the instruction and is sign extended in **ID**
- The value that goes into `$s1` is computed in **EX** and written in **WB**

(b) `lw $s1, 400($s2)`

- The value in `$s2` is read in **ID**
- The immediate value `400` comes from the instruction and is sign extended in **ID**
- The effective address $\$s2 + 400$ is worked out in **EX**
- The value loaded into `$s1` is read from memory in **MEM** and written in **WB**

(c) `sw $s1, 400($s2)`

- The value in `$s1` is read in **ID** and stored to memory in **MEM**
- The value in `$s2` is read in **ID**
- the immediate value `400` comes from the instruction and is sign extended in **ID**
- The effective address $\$s2 + 400$ is worked out in **EX**

(d) `beq $s1, $s2, LABEL`

- The value in `$s1` is read in **ID**
- The value in `$s2` is read in **ID**
- The branch constant for `LABEL` comes from the instruction and is sign extended in **ID**
- The branch target address is worked out in **EX**
- The equality check is done in **EX**
- Nothing gets written to a register

2.

if the instructions become

`lw $s1, $s2` and `sw $s1, $s2`,

then we don't really need the ALU to add an offset anymore. The address is just the value already sitting in the base register.

so for these memory instructions, the EX stage is no longer needed.

$$\texttt{lw} : IF \to ID \to MEM \to WB$$
$$\texttt{sw} : IF \to ID \to MEM$$

so the pipeline is shorter by one stage for these instructions.

What happens to dependencies:

- A load now gives its value one stage earlier than before, because there is no EX stage before the memory read
- That means load use hazards are a little better, and sometimes you need fewer stalls
- A base register used by `lw` or `sw` is no longer needed for address calculation in EX. Its value just moves from ID to MEM
- So dependencies involving the base register are also easier to handle

3.

(i)

I_1 : `lw $s1, 40($s6)`
I_2 : `add $s6, $s2, $s2`
I_3 : `sw $s6, 50($s1)`

(a) all data dependencies

- $I_1 \rightarrow I_3$ on `$s1`. The load writes `$s1`, and the store uses it as its base register
- $I_2 \rightarrow I_3$ on `$s6`. The add writes `$s6`, and the store uses it as the value to store

(b) No forwarding

without forwarding, I_3 has to wait until both earlier instructions write back

```
lw $s1, 40($s6)
add $s6, $s2, $s2
nop
nop
sw $s6, 50($s1)
```

(c) full forwarding

with full forwarding, both dependencies can be handled by forwarding:

- the loaded `$s1` can be forwarded to the store address path
- the new `$s6` can be forwarded to the store data path

so no `nop` is actually needed

```
lw $s1, 40($s6)
add $s6, $s2, $s2
sw $s6, 50($s1)
```

(ii)

I_1 : `lw $s5, -16($s5)`
I_2 : `sw $s5, -16($s5)`
I_3 : `add $s5, $s5, $s5`

(a) All data dependencies

- $I_1 \rightarrow I_2$ on `$s5`. The store uses the new value both as the base address and as the store data.
- $I_1 \rightarrow I_3$ on `$s5`.

(b) no forwarding

without forwarding, the store has to wait for the load to write back

```
lw $s5, -16($s5)
nop
nop
sw $s5, -16($s5)
add $s5, $s5, $s5
```

(c) full forwarding

even even with full forwarding, there is still one load use hazard from I_1 to I_2. the loaded value is not ready soon enough for the very next store

so one `nop` is still needed

```
lw $s5, -16($s5)
nop
sw $s5, -16($s5)
add $s5, $s5, $s5
```

After that one stall the value can be forwarded :)

4. Dependencies, forwarding, and stalls

I_1 : `add $s3, $s4, $s2`
I_2 : `sub $s5, $s3, $s1`
I_3 : `lw $s6, 100($s3)`
I_4 : `add $s7, $s3, $s6`

All RAW dependencies

- $I_1 \rightarrow I_2$ on `$s3`
- $I_1 \rightarrow I_3$ on `$s3`
- $I_1 \rightarrow I_4$ on `$s3`
- $I_3 \rightarrow I_4$ on `$s6`

Handled by forwarding

- $I_1 \rightarrow I_2$ on `$s3`
- $I_1 \rightarrow I_3$ on `$s3`

- $I_1 \to I_4$ on `$s3`

Still causes a stall

- $I_3 \to I_4$ on `$s6`, which is the usual load use hazard

so one stall is basically needed between I_3 and I_4

5.

(a) Circuit to predict taken in IF

If the current instruction is a branch, the predicted next PC should be

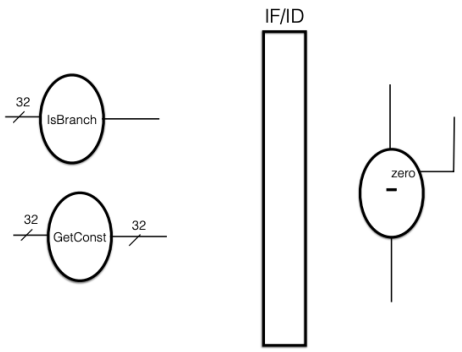
$$(PC + 4) + (\texttt{GetConst}(\text{Instr}) \ll 2).$$

If the current instruction is not a branch, the next PC should be $PC + 4$.

So the idea is pretty much:

1. find $PC + 4$
2. use `GetConst` on the instruction
3. shift that value left by 2
4. add it to $PC + 4$
5. use a MUX controlled by `IsBranch`

5. Suppose we wish the architecture to implement branch prediction that assumes that branch instruction conditionals evaluate to true. You may assume the existence of two special circuits that take in as input the instruction and can return a result quickly enough that they can be applied within the IF stage:
   - IsBranch looks at the OpCode and returns 1 if the instruction is a branch instruction. For the purposes of this problem, you may assume that the only branch instruction we are dealing with is beq.
   - GetConst takes in the instruction and returns as a 32-bit signed binary what is potentially the 16-bit constant (i.e., it simply pulls out the 16 lowest order bits of the instruction and sign-extends the result).



the the PC feeds instruction memory. The fetched instruction goes to `IsBranch` and `GetConst`. Then the shifted branch constant is added to $PC + 4$. After that, a MUX chooses between $PC + 4$ and the predicted branch target.

(b)(i)

Since we are predicting that the branch is true, the processor starts fetching from the branch target right away.

If the condition later turns out to be false, the machine has to recover by going to the fall through instruction That address is the branch instruction's own

$$PC + 4.$$

So the main extra piece of information we need to carry through the pipeline is:

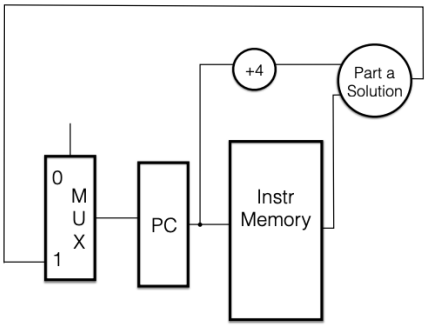
- the fall through address $PC + 4$ for the branch

it also helps to carry a bit saying that this instruction was treated as a predicted branch, so the control logic knows when a false result means the prediction was wrong

(b)(ii)

to recover when the branch is actually false, we need:

1. a path that carries the branch's saved $PC + 4$ forward through the pipeline
2. branch check logic in the stage where the condition is actually tested
3. control back to the MUX before the PC, so the saved $PC + 4$ can be chosen when the prediction was wrong



Note that in your solution the +4 circuit might have been embedded inside your part (a) - it has been pulled out here for a reason...

so basically, the behavior is:

- normally, the PC gets the predicted target from part (a)
- if the branch later turns out to be false, the MUX picks the saved $PC + 4$